# Towards a WasmGC backend and runtime for the MicroHs compiler

Master's thesis part 1: project proposal

**Martijn Voorwinden – 7854064**
Utrecht University – Department of Information and Computing Sciences
1st supervisor: Marco Vassena
2nd supervisor: Wouter Swierstra
Daily supervisor: Apoorva Anand

# Abstract

*Today, porting Haskell code to web browsers is complicated. Current approaches such as compilation to JavaScript or WasmMVP suffer from issues such as large binary sizes or performance issues. This is due to semantic differences between the target language and Haskell, as well as having to ship a custom GC in case of WasmMVP. A promising alternative is compiling Haskell to WebAssembly with GC support (WasmGC), which allows compilers to make use of the host environment's GC instead of shipping their own, which can lead to smaller binary sizes and improved maintainability. The goal of this project is to investigate the effects of using WasmGC as a target for Haskell compilers by extending the MicroHs compiler with a new backend and runtime system. Since WasmGC is not a language inteded to be written by hand, we design a custom DSL for generating WasmGC code to help in the implementation of the runtime system. We evaluate the generated Wasm binaries in terms of binary size and runtime performance against the existing MicroHs Wasm backend and native backend. Preliminary results show that the WasmGC backend produces significantly smaller binaries than both the existing Wasm backend and the native backend, and that the WasmGC backend performs significantly better than the existing Wasm backend. We discuss the plan going forward for Part 2 of the thesis, which includes completing the implementation of the WasmGC backend and runtime system, more extensive evaluation of the produced WasmGC binaries, and the development of a demo application to showcase the capabilities of the WasmGC backend.*

# Contents

**1 Introduction** **1**

**2 Background** **3**
2.1 MicroHs . . . 3
2.2 WebAssembly . . . 4

**3 Compiler and Runtime System Design** **8**
3.1 Compiler and Runtime System Overview . . . 8
3.2 Domain Specific Language . . . 8
3.2.1 Abstract Representation . . . 8
3.2.2 Code Generation . . . 8
3.2.3 Monadic Interface . . . 9
3.3 WasmGC Backend Implementation . . . 9
3.4 Runtime System Implementation . . . 10

**4 Preliminary Results** **14**

**5 Related Work** **17**
5.1 Haskell for the Web . . . 17
5.2 WebAssembly with Garbage Collection . . . 17
5.3 Combinator Based Compilers and Runtime Systems . . . 18

**6 Plan** **19**

**Bibliography** **22**

**Appendices** **23**
A: Ethics and Privacy QuickScan . . . 24

# Introduction

Today, porting Haskell code to web browsers is complicated. Current approaches such as transpilation to JavaScript [1] suffer from performance issues and lack of semantic similarity (e.g. functional vs imperative, lazy vs eager eveluation, etc.), while solutions such as containerization can not be used in browser environments.

A promising alternative is compiling Haskell to WebAssembly (Wasm) [2, 3, 4], a low-level bytecode format designed for efficient execution in portable environments such as web browsers, but also for non-browser engines which can be used for server-side applications. Wasm however has its own challenges, the original version of Wasm (WasmMVP) does not natively support garbage collection (GC), which forces compilers targeting WasmMVP to ship their own memory management systems.

This changed with the recent release of Wasm 3.0 [5] due to the integration of the WasmGC proposal, which extends Wasm with user-defined garbage collected (GC) data types. This allows compilers targeting Wasm to make use of the host environment's GC instead of shipping their own, which can lead to smaller binary sizes and improved maintainability. This is due to the fact that runtime systems can be simpler and smaller as they do not need to include a GC implementation.

Despite the potential benefits of using WasmGC, current Wasm backends of compilers such as GHC's Wasm backend [3] do not make use of it, and instead target WasmMVP.

**Contributions** The goal of this project is to investigate the performance tradeoffs of using WasmGC as a target for Haskell compilers. To this end we will extend the Haskell compiler MicroHs [6] with a WasmGC backend. Concretely, we will do the following:

1. Design and implement a custom domain specific language (DSL) for generating WasmGC code, for use in the runtime system and backend of the compiler. This will help us to write more maintainable and readable code compared to writing raw WasmGC.

2. Design and implement a runtime system for the WasmGC backend using the custom DSL.

3. Evaluate the produced Wasm binaries in terms of size and runtime performance against other Haskell-to-Wasm compilers and MicroHs's existing C backend.

**Previous Work** Previously, Anand [7] made a prototype extension to MicroHs to target WasmGC and implemented a proof-of-concept runtime system written in a combination of WasmGC and a custom DSL. Initial experiments showed promising results in terms of binary size, however runtime performance was not tested. Our work will extend this work with a new DSL capable of generating the full runtime system instead of just parts of it, building a more complete runtime system that supports more Haskell features, and evaluating the runtime performance of the generated Wasm binaries.

**Document Structure** The rest of this thesis proposal is structured as follows:

- Chapter 2 provides the reader with the necessary background on WebAssembly and the architecture of MicroHs.
- Chapter 3 describes the design and implementation of the WasmGC backend and runtime system.
- Chapter 4 describes preliminary results on the runtime performance of the generated Wasm binaries.
- Chapter 5 discusses related work on the topic.
- Chapter 6 outlines the plan for Part 2 of the thesis.

# Background

## 2.1 MicroHs

MicroHs [6, 8] is a minimal Haskell compiler written in Haskell, with a runtime system written in C. MicroHs follows a standard compilation pipeline, where it starts by parsing the Haskell source code, then type-checking it, desugaring it into lambda terms, optimizing the lambda terms, and finally transforming the lambda terms into combinators through a process called bracket abstraction. MicroHs converts the combinators into C code, which is then combined with the runtime system into a single C codebase. This codebase can then be compiled with standard C compilers to obtain a native executable, or to Wasm using Emscripten [9]. The compilation pipeline is illustrated in Figure 2.1.

**Combinators** MicroHs uses a combination of combinators and primitive operations to represent compiled Haskell code. SKI combinators are a small set of combinators that can be used to represent any lambda term. The three basic SKI combinators are:

$$
\begin{aligned}
S\ x\ y\ z &= x\ z\ (y\ z) \\
K\ x\ y &= x \\
I\ x &= x
\end{aligned}
$$

Further combinators include those used by Turner [10] or are added by MicroHs itself to improve performance of the generated code.

Primitive operations can be divided into two types: strict primitives and non-strict primitives. Strict primitives require all of their arguments to be reduced to weak head normal form (WHNF) before they can be applied, while non-strict primitives can be applied without reducing their arguments first. WHNF in the context of combinators means that the argument is either a primitive value (e.g. an integer) or a combinator or primitive application with to few arguments to be reduced. Examples of strict primitives include arithmetic operations such as addition and multiplication, while examples of non-strict primitives include the monad operations for the IO monad (return and `>>=`).

**Representing Combinators** To efficiently represent combinators and their arguments, MicroHs uses a graph representation. Each node can either be a leave node storing a combinator or a primitive value, or an internal node storing a reference to its left and right children. This allows for efficient sharing of subterms and avoids the need for copying when reducing combinators. For example, the combinator application `K (I x) y` can be represented as the graph shown in Figure 2.2.

**Runtime System** After building the graph representation of the combinators, the runtime system is responsible for reducing the graph to WHNF. This is done by repeatadly walking the left spine of the graph until a leaf storing a combinator is reached,

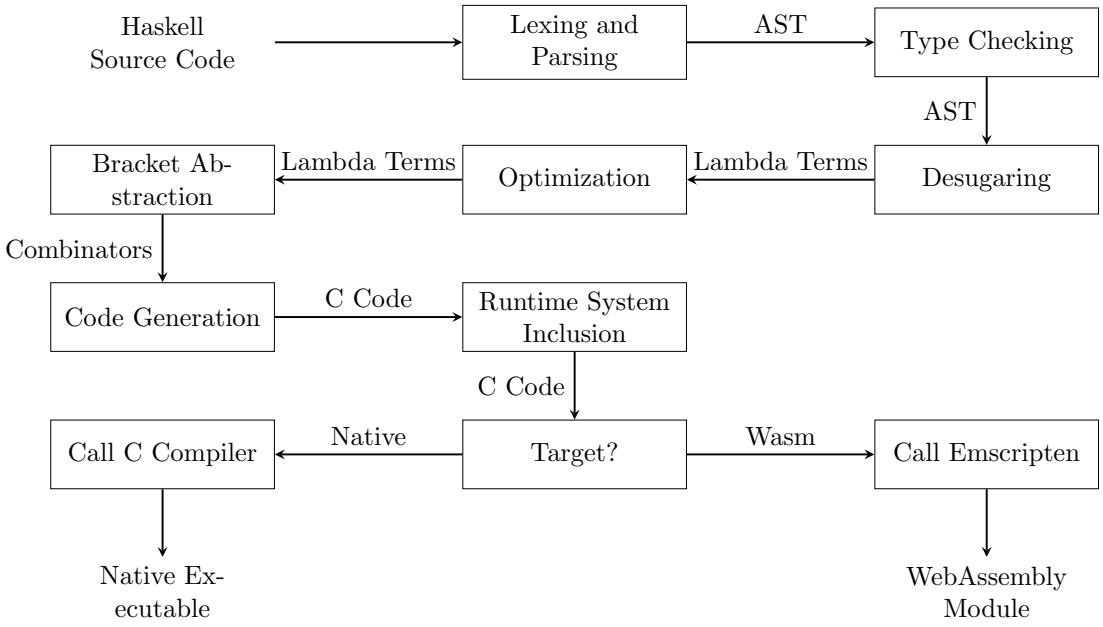


Figure 2.1: MicroHs Compilation Pipeline

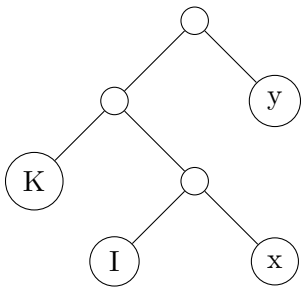


Figure 2.2: Graph representation of the combinator application `K (I x) y`

at which point the reduction rules for that combinator are applied. Since the reduction rules for most combinators involve multiple arguments, the runtime system maintains a left ancestor stack (LAS) to keep track of the ancestors of the current node (the left spine of the graph). This allows the runtime system to efficiently access the arguments needed for reduction without having to traverse the graph multiple times. An example reduction for the combinator application `K (I x) y` is shown in Figure 2.3.

We can use the algorithm shown in Algorithm 1 to perform the reduction. The `step` function takes the current node, the left ancestor stack, and the left ancestor stack pointer as input and returns the new current node and the updated left ancestor stack pointer after performing one reduction step. The algorithm continues until the current node is fully reduced (i.e. it is a leaf storing a primitive value or an application with to few arguments to be reduced).

## 2.2 WebAssembly

Wasm is a binary stack-based instruction format. Wasm programs run in an isolated environment by construction. They can be interacted with from outside of the environment through imports and exports, this allows Wasm modules to call (imported) functions such as printing to the console, and allows a host environment to initiate the

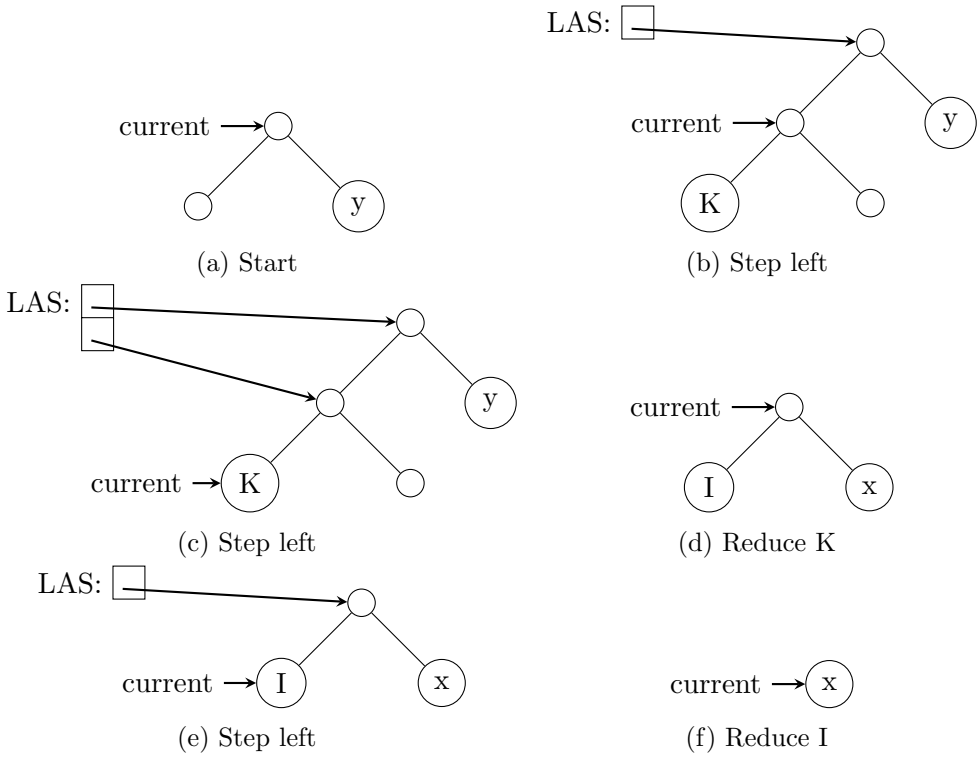


Figure 2.3: Reduction of the combinator application `K (I x) y`

execution of a Wasm module by calling an exported function. The programs themselves are organized into modules containing functions, memory, imports/exports, and other components.

Wasm has a text format called WAT (WebAssembly Text format) which is a human-readable representation of the binary format. From now on we will use WAT syntax to illustrate Wasm concepts. Using WAT a simple list of instructions that adds two numbers looks like this:

```
(i32.const 1)
(i32.const 2)
(i32.add)
```

The first two instructions push the constants 1 and 2 onto the stack, and the last instruction pops the two values from the stack, adds them together, and pushes the result (3) back onto the stack.

**Instructions and Functions** Instructions in Wasm are the basic building blocks of computation, including simple operations like `i32.add` for arithmetic and `local.get` and `local.set` for variable access. Control flow uses structured instructions such as `if`, `block`, `loop`, and `br` (branch), where conditions are represented as integers (0 is false, non-zero is true). Branching to a block label continues execution after the block end, while loop labels restart at the loop beginning.

Functions are blocks of code that can take parameters, return results, and contain local variables. They are defined using the `func` keyword and can be called with the `call` instruction.

An example piece of wasm code that demonstrates control flow and function calls is shown below:

**Algorithm 1:** Graph reduction algorithm based on [10].

**Input :** Node `n` to reduce
**Output:** Reduced node
$las \leftarrow \emptyset$;
$las_{pointer} \leftarrow -1$;
$cur \leftarrow n$;
**while** *cur is not fully reduced* **do**
  **if** *cur is a combinator* **then**
    $(cur, las_{pointer}) \leftarrow \text{step}(cur, las, las_{pointer})$;
  **else**
    $cur \leftarrow \text{leftChild}(cur)$;
  **end**
  $las[las_{pointer}] \leftarrow cur$;
  $las_{pointer} \leftarrow las_{pointer} + 1$;
**end**

```
;; import a print function
(import "env" "print" (func $print (param i32)))

(block $exit
  (loop $loop
    (local.get $counter)
    (i32.const 0)
    (i32.eq)
    (br_if $exit) ;; exit if counter == 0
    (local.get $counter)
    (i32.const 1)
    (i32.sub)
    (local.set $counter)
    (local.get $counter)
    (call $print)
    (br $loop) ;; continue looping
  )
)
```

**Types** Wasm's type system consists of value types and heap types. Value types include primitive numeric types such as `i32` (32-bit integer) and `i64` (64-bit integer), and reference types.

Reference types are references to heap types such as functions, structs, and arrays. The references are typed, meaning that every reference has a specific type associated with it. `anyref` is a supertype of all reference types in Wasm, allowing it to store references to any type of object, but it requires runtime type casts when accessing these fields. A cast can be performed using the `ref.cast` instruction, which casts a reference to a different type and traps if the cast is invalid. The `ref.test` instruction checks if a reference can be cast to a given type. Reference types can also be nullable, meaning that they can point to either an object of the specified type or null. The `ref.null` instruction creates a null reference of a given type, and the `ref.is_null` instruction checks if a reference is null.

Custom types can be defined in Wasm using the `type` keyword. These custom types can be used to define more complex data structures such as structs and arrays. Structs are collections of named fields, each with their own type, while arrays are collections of elements of the same type. For example, we can define a struct type for a binary tree node as follows:

```
(type $Node (struct
  (field $left  (mut anyref))
  (field $right (mut anyref))
))
```

In this definition, we use the `anyref` type for the fields, which allows us to store references to any type of object in the `left` and `right` fields of the `Node` struct. The `mut` keyword indicates that the fields are mutable, meaning that their values can be changed after the struct is created.

# Compiler and Runtime System Design

## 3.1 Compiler and Runtime System Overview

If you recall the architecture of the MicroHs compiler from Chapter 2, the compiler transforms the generated combinators to a useable format for the C runtime, after which it combines the generated code with the runtime system and compiles it to a binary.

To target WasmGC, we will need to expand on these two steps. Next to generating the C-code for the combinators, we will also generate WasmGC code for constructing the combinator graph. Likewise, next to the C runtime system, we will also implement a runtime system in WasmGC.

## 3.2 Domain Specific Language

Since WasmGC is a low-level language generally meant to be targeted by compilers instead of being written by hand, writing the runtime system directly in WasmGC would be difficult and error-prone. To solve this issue, we have designed and implemented a simple DSL embedded in Haskell for generating WasmGC code, which abstracts away the explicit stack manipulation that is necessary when writing Wasm code directly. The DSL is an imperative language and consists of three main components: an abstract representation of programs, a code generation function that takes a program in the abstract representation and generates WasmGC code, and a monadic interface for writing programs in the abstract representation.

### 3.2.1 Abstract Representation

The abstract representation of programs in the DSL consists of a set of data types that represent the different constructs in the language. The main data type is `RtMod`, which represents a module in Wasm and contains the type definitions, imports, and functions of the module. The type definitions `RtTypDef` consist of a name and what type to bind it to, while functions `RtFunc` have explicitly declared local variables at the function definition level and a body that consists of a list of expressions. Expressions `RtExpr` can be variable assignments, function calls, control flow constructs such as if statements and while loops, and other constructs such as literals and variable references. The abstract representation allows us to write programs in a more high-level and structured way compared to writing raw WasmGC code, while still being close enough to WasmGC to allow for simple code generation.

### 3.2.2 Code Generation

For generating the WasmGC code from the AST, we make use of the `DSL` monad, which has a state that has a generator for generating fresh variable names, settings such as

the indent level for pretty printing, and a list of types which we use for simple type checking. We can than generate a wasm program using the function `toWat :: String → RtMod → DSL String`. This function takes a program represented as a `String` and a runtime system represented as a `RtMod` and generates the corresponding WasmGC code as a `String`. The generated code is in the human-readable WAT format, which can then be compiled to Wasm using a tool such as `wat2wasm`.

### 3.2.3 Monadic Interface

The monadic interface allows us to write programs in an imperative style by using `do` notation while the DSL is embedded in a pure functional language. For this we define two monads, namely `WasmModM` for defining modules and `WasmM` for defining functions. The `WasmModM` monad has three helper functions, these are `type_ :: String → RtTyp → WasmModM ()` for defining types, `import_ :: String → [RtTyp] → [RtTyp] → WasmModM ()` for defining imports, and `register → RtFunc → WasmModM ()` for registering functions. These functions can than be used as follows to define a module:

```
myModule = runWasmModM $ do
    type_ "Node" $
        record [ ("left" , noderef)
               , ("right", noderef) ]
    import_ "add" [i32, i32] [i32]
    register someFunc -- someFunc is a function defined elsewhere in
                      -- the code using the WasmM monad
```

For defining functions, we make use of the following helper functions: *input*, *output*, and *var* for defining the input parameters, output parameters, and local variables of a function respectively. Furthermore, we have the *uses* function for indicating that a function makes use of another function, this returns a partially applied `Call` constructor that can be used later. Last, we have functions for control flow constructs: *when* for if statements, *if_* for if-else statements, *switch* for switch statements, and *while* for while loops. To make the syntax easier to read, we have also defined `then_` and `else_` functions to be id, this way we can write code like the following:

```
someFunc = runWasmM $ do
    x   <- input "x"   i32
    output i32
    y   <- var   "y"   i32
    add <- uses  "add"

    -- `gets` retrieves the value of a variable,
    -- `i` is a helper function for creating integer literals
    if_ (gets x .== i 0) $ do
        (then_ $ do
            y =: i 1)
        (else_ $ do
            y =: add [gets x, gets x])
    return_ (gets y)
```

## 3.3 WasmGC Backend Implementation

The abstract representation of expressions in the MicroHs compiler is defined as follows:

```
data Exp
    = Var String
    | Lit Lit
    | App Exp Exp
    | Lam Ident Exp
```

To generate WasmGC code from this representation, we will need to map the abstract representation to the correct code to construct a graph in our DSL. This is done by the function `compileExp :: Exp → RtExpr`, which takes an expression in the abstract representation and generates the corresponding code in our DSL.

```
compileExp :: Exp -> RtExpr
compileExp (Var x)          = gets "functions" .!! i tIdx
    where tIdx = read (tail (showIdent v)) :: Int
compileExp (Lit (LPrim p)) = i' $ case primIndex p of
    Just idx -> idx
    Nothing  -> error "Unknown primitive"
compileExp (Lit (Lint n))  = mkValue $ i' n
compileExp (App e1 e2)     = mkNode (compileExp e1) (compileExp e2)
compileExp (Lam _ _)       = error "Lambdas should not exist"
```

If we encounter a variable, we generate the instructions to lookup the variable in the `functions` array (the functions array contains references to nodes representing different initialized and to be initialized functions) and return a reference to the variable in question. In case of a literal that is a primitive or combinator we look up its index in the primitive table and return it as a reference to the index. For other literals such as integers, we generate instructions to box the literal in a `Value` type. Applications are done by constructing a new node with the two arguments as its children. Lambdas should be eliminated by the time we reach the WasmGC backend, so we can ignore them.

## 3.4 Runtime System Implementation

The runtime system implements the algorithm discussed in Chapter 2 for performing graph reduction. The runtime system is implemented in WasmGC using our DSL and consists of multiple types and functions.

**Types** For the graph representation, we use a struct named `Node` with two fields `left` and `right` which are references to either other nodes, combinators/primitives, or values (we make use of `anyref` as it is the common supertype of all references). For the LAS, we use an array of references to nodes, and lastly for values we use a struct named `Value` with a field `val` of type `anyref` to store primitive or fully reduced values. The type definitions for these types are as follows:

```
module_ $ do
    type_ "Node" $
        record [ ("left" , anyref)
               , ("right", anyref) ]

    type_ "Stack" $
        array noderef
```

```
    type_ "Value" $
        record [ ("val", anyref) ]

    ...
```

**Functions** For the functions, we have a main function that serves as the entry point for the program, a `reduce` function that implements the graph reduction algorithm, and a `step` function that applies the reduction rule for a given combinator or primitive. The main function is responsible for constructing the initial graph and starting the reduction process:

```
main $ do
    -- used by the graph construction code
    node      <- var "node" noderef
    n         <- var "n"    i32
    functions <- var "functions" stackref
    -- reduction should return a value
    output valueref
    -- the main function makes use of the reduce function
    reduce    <- uses "reduce"

    -- initialize temporary functions array
    functions =: mkStack (i fs) -- fs is the number of functions in the program
    while (gets n .< i fs) $ do
        -- fill the functions array with empty nodes (I 0)
        setAt (gets functions) (gets n) (mkNode (i' iComb) (i' 0))
        n =: increment (gets n)
    n =: i 0
    -- the graph construction code should be inserted here
    insertProgram
    -- since the program we insert produces a node on the stack,
    -- we need an assignment to the node variable without a right hand side
    node      =: ignore
    -- we free the functions stack here to save some memory
    functions =: nil stackref
    -- start the reduction process
    return_ (reduce [gets node])
```

The `reduce` function implements the graph reduction algorithm as discussed in Chapter 2. It initializes an array to be used as the LAS and an index (`sptr`) to keep track of the top of the LAS. It then enters a loop where there are four cases to consider. Case 1: if the current node is `null`, then we have finished reduction and there was no result. Case 2: if the current node has a left child that is the `I` combinator and a right child of the type value, then we have finished reduction and the result is the value stored in the right child. Case 3: if the current node has a left child that is a combinator or primitive, then we call the `step` function to apply the reduction rule for that combinator or primitive. Case 4: if the current node has a left child that is not a combinator or primitive, then we walk down the left spine of the graph by setting the current node to its left child and pushing it onto the LAS. The code for this is as follows:

```
function "reduce" $ do
    root    <- input "root"  noderef
    output valueref
    step    <- uses "step"
    stack   <- var "stack"   stackref
    sptr    <- var "sptr"    i32
    current <- var "current" noderef

    -- initialize the LAS, sptr, and current node
    stack   =: mkStack (i 100000) -- we use a fixed maximum size for the LAS
    sptr    =: i (-1)
    current =: gets root
    -- the reduction loop
    while true $ do
        -- Case 1: if current is null, we are done and there was no result
        when (isNull (gets current)) $ do
            return_ (nil valueref)
        -- Case 2: if current.left is an i31ref with value I(8) and current.right
        --         is of type Value, we have reached a result
        when (getLeft (gets current) `is` i31ref) $ do
            when ((asI32 (getLeft (gets current)) .== i iComb) .&&
                    (getRight (gets current) `is` valueref)) $ do
                return_ (getRight (gets current) `as` valueref)
        -- add current to the LAS and increment sptr
        sptr =: increment (gets sptr)
        setAt (gets stack) (gets sptr) (gets current)
        if_ (getLeft (gets current) `is` i31ref)
        -- Case 3: if current.left is an i31ref we can apply a reduction rule
            (then_ $ do
            [current, sptr] ==: step [gets current, gets stack, gets sptr])
        -- Case 4: otherwise we continue down the left spine of the graph
            (else_ $ do
                current =: (getLeft (gets current) `as` noderef))
    -- this return is unreachable but we need it to satisfy the type checker
    return_ (nil valueref)
```

The `step` function has a case for each combinator and primitive, which apply the reduction rule for the combinator/primitive in question. For this is makes use of metavariables to represent arguments extracted from the LAS, this makes the reduction rules look closer to the mathematical definitions of the combinators and primitives. The code for the `step` function is as follows:

```
function "step" $ do
    node  <- input "node"  noderef
    stack <- input "stack" stackref
    sptr  <- input "sptr"  i32
    output noderef
    output i32
    -- the combinator/primitive to reduce
    comb  <- var "comb" i32
    -- the new current node and sptr after reduction
```

```
    current <- var "current" noderef
    i_      <- var "i"       i32
    -- the metavariables
    x <- var "x" anyref
    y <- var "y" anyref
    z <- var "z" anyref
    w <- var "w" anyref
    v <- var "v" anyref

    -- extract the combinator/primitive from the node
    comb =: asI32 (getLeft (gets node))
    -- lookup the combinator/primitive and apply the reduction
    switch (gets comb) prims
    -- return the new current node and sptr after reduction
    return_ (Block [ Var current, Var i_ ])
```

We make use of the helper function `prims` that generates the cases for each combinator and primitive based on a primitive table that contains the reduction rules for each combinator and primitive. The reduction rules are defined using a helper function `redRule` that takes a left node definition, a right node definition, and the amount of arguments to pop from the LAS. Using this helper function, we can easily define reduction rules such as the following rule for the `B` combinator $B\ x\ y\ z = x\ (y\ z)$:

```
redRuleB = redRule ln rn n
    where
        ln = x
        rn = mkNode y z
        n  = 3
```

**Module** Using the types and functions defined above, we can define the complete module for the runtime system as follows:

```
module_ $ do
    -- types defined earlier
    ...
    -- we need to import a print function for some primitives
    import_ "print" [i32] []
    -- register the functions defined earlier
    register reduceFunc
    register stepFunc
    -- register the main function as the entry point for the program
    registerMain mainFunc
```

# Preliminary Results

Since existing Wasm GCs may not perform well on memory workloads typical of functional programs, we have developed an experiment to quickly determine if our WasmGC port is viable. The simple benchmark program below (inspired by the |Lambda-Calculus cooked n-ways— benchmark [11]) is designed to generate a large number of substitutions.

The program defines a simple Peano number type and implements addition, multiplication, factorial, and equality. It then computes the factorial of 6 (720) and the sum of the first 37 natural numbers (703) plus 17 (720), after which it checks if the two results are equal. The primitive `pint32` prints an integer to the console, so the program will print 1 if the results are equal and 0 otherwise.

```
module Simple where
import qualified Prelude ()
import Primitives

data Peano = Zero | Succ Peano
    deriving ()
data Bool = False | True
    deriving ()

one :: Peano
one = Succ Zero
two :: Peano
two = Succ one
three :: Peano
three = Succ two
add :: Peano -> Peano -> Peano
add Zero     y = y
add (Succ x) y = Succ (add x y)
mul :: Peano -> Peano -> Peano
mul Zero     _ = Zero
mul (Succ x) y = add y (mul x y)
fac :: Peano -> Peano
fac Zero     = Succ Zero
fac (Succ x) = mul (Succ x) (fac x)
eqnat :: Peano -> Peano -> Bool
eqnat Zero     Zero     = True
eqnat Zero     (Succ _) = False
eqnat (Succ _) Zero     = False
eqnat (Succ x) (Succ y) = eqnat x y
sumto :: Peano -> Peano
```

```
sumto Zero     = Zero
sumto (Succ x) = add (Succ x) (sumto x)
n5 :: Peano
n5 = add two three
n6 :: Peano
n6 = add three three
n17 :: Peano
n17 = add n6 (add n6 n5)
n37 :: Peano
n37 = Succ (mul n6 n6)
n703 :: Peano
n703 = sumto n37
n720 :: Peano
n720 = fac n6
boolToInt :: Bool -> Int
boolToInt False = 0
boolToInt True  = 1

main :: IO ()
main = (_primitive "pint32") (boolToInt (eqnat n720 (add n703 n17)))
```

The results of the benchmark are shown in Table 4.1. We compare three different metrics across three different backends: execution time, memory usage, and binary size. The execution time is the minimum execution time out of 1000 runs. Due to the timer used in the native backend, the time always results in 1ms as the timer only has millisecond precision. Memory usage is the resident set size (RSS) of the process after execution and only measured for the Wasm backends.

As can be seen the WasmGC backend performs significantly better than the existing Wasm backend. Comparing the run time to the native backend is not very meaningful due to the low timer resolution, thus we cannot say if the WasmGC backend is faster or slower than the native backend.

The memory usage of the WasmGC backend is significantly lower than the existing Wasm backend. This is likely due to the fact of a memory leak in the module generated by emscripten, as the memory usage is lower when the benchmark is run for a lower amount of runs, while the memory usage of the WasmGC backend is stable across different amounts of runs.

The binary size of the WasmGC backend is also significantly smaller than the existing Wasm backend and the native backend, this has two main reasons: the WasmGC backend does not need to include a GC implementation in the binary, and the WasmGC backend does not offer support for the full set of primitives the other backends support, which means that the runtime system is significantly smaller.

| **Backend** | **Time (ms)** | **Memory Usage (KiB)** | **Binary Size (B)** |
|---|---:|---:|---:|
| WasmGC | 0.991 | 65 640 | 4 574 |
| Wasm (Emscripten) | 2.250 | 845 232 | 452 264 |
| Native | 1.000 | N/A | 216 184 |

Table 4.1: Results of the benchmark comparing the WasmGC backend against the existing Wasm backend (through emscripten) and native backend. For the Wasm backends the time is the minimum execution time out of 1000 runs. Memory usage is the resident set size (RSS) of the process after execution and only measured for the Wasm backends.

# Related Work

## 5.1 Haskell for the Web

Historically, running Haskell code in web browsers has been achieved using GHCJS [1], a compiler that transpiles Haskell code to JavaScript. While GHCJS allows Haskell code to run in browsers, it suffers from multiple problems. Due to it having to include large parts of the GHC runtime system resulting in large bundle sizes [12]. Furthermore, the development has lagged behind GHC itself, causing GHCJS to not be able to take advantage of the latest compiler optimizations and language extensions [1].

WebAssembly has emerged as a promising alternative for running Haskell code on the web. Originally through the Asterius project [4], and more recently through GHC's own Wasm backend [3]. Programs can also be compiled to Wasm by MicroHs, by first compiling to C and then using emscripten [9] to compile the C code to Wasm. However, all of these approaches target traditional Wasm without GC support. This means they have to include a memory management system in the produced Wasm binaries, leading to larger binary sizes.

Other functional programming languages that share a large amount of similarities with Haskell have also been used for web development. Elm [13] is a purely functional programming language that compiles to JavaScript and is designed specifically for building web applications. Similarly, PureScript [14] is another strongly-typed functional programming language that compiles to JavaScript with a syntax and semantics inspired by Haskell. While they may look like Haskell, there are many semantic differences and are thus not directly applicable to this thesis's goals.

## 5.2 WebAssembly with Garbage Collection

WebAssembly's GC support is a relatively recent addition to the WebAssembly specification, aimed at improving the performance and efficiency of languages that rely on GC, such as Haskell, Java, and C#. Since its introduction, it has seen wide adoption in various projects and languages. It is supported by all major web browsers, including Chromium-based browsers, Firefox, and Safari, as well as non-web environments like Node.js, Deno and Wasmtime [5]. Furthermore there is support for GC specific optimizations in popular Wasm optimzers such as Binaryen [5, 15]. And it is targeted by multiple compilers, such as Dart, Kotlin/Wasm, WasOCaml, Hoot, and J2CL/Wasm [16, 17, 18, 19, 20, 21].

As demonstrated by the V8 team, using Wasm's with GC is highly optimizable, both by compile time optimizations as well as runtime optimizations [22]. They demonstrated that using a compile time optimizer such as Binaryen with Wasm with GC can produce binaries that have up to 2 times higher throughput (higher is better). Likewise, they demonstrated that enabling runtime optimizations in V8 can lead to lower latency (lower is better) by porting the Google Sheets Calc Engine to Wasm with GC.

Performance testing of binaries produced by the Kotlin compiler shows that the Wasm backend produces binaries that are significantly faster than the JavaScript backend, and is approaching the performance of the JVM backend [17]. When looking at functional programming languages, the WasOCaml compiler targets Wasm with GC, per their benchmarks, binaries produced by WasOCaml are slower by an almost constant factor of circa 2 times when compared to native OCaml binaries [18]. For Haskell specifically, the GHC Wasm backend has been benchmarked against GHC's native backend, showing that the Wasm backend is between 4 to 10 times slower than the native backend depending on the benchmark, most likely due to not using WasmGC and other modern Wasm features [21].

## 5.3 Combinator Based Compilers and Runtime Systems

The combinator based compiler backend and runtime MicroHs uses is based on the work of Turner [10], who introduced the idea of compiling programs to SKI combinators, combined with his own combinators, and strict primitives, and executing them on a graph-reduction machine. Turner made use of this system in multiple functional programming languages, such as the SASL and KRC languages which were intended for research and teaching purposes [23, 24]. Later, the system was also used in the production Miranda language [25], which has had significant influence on the design of Haskell.

While modern runtimes for lazy functional programming languages such as GHC are implemented using abstract machines such as the Spineless Tagless G-machine (STG) [26], experimenting with combinator based runtimes is still relevant as our focus is not on optimizing the runtime system itself, but rather on exploring the effects of using Wasm GC support as opposed to shipping our own GC when compiling Haskell to Wasm.

# Plan

In part two of the thesis, we will complete the implementation of a WasmGC backend and runtime system capable of running a set of standard Haskell benchmarks. We will then measure and evaluate the performance of the generated Wasm code, and develop a small demo application to showcase the capabilities of the WasmGC backend. We will aim to follow the following timeline for part two:

1. **Weeks 1-10:** Implement the WasmGC backend and runtime system.
2. **Weeks 11-15:** Set up the benchmarks, measure tooling, and perform performance evaluation.
3. **Weeks 16-20:** Develop a demo application and/or implement optimizations based on the results of the performance evaluation and the time remaining.
4. **Weeks 21-24:** Write auxiliary sections of the thesis, prepare the defense, and catch up on any remaining work (in case of delays). If time permits, explore additional features such as lightweight threads and JavaScript interoperability.

In case of major delays during step one or two, we can adjust the plan to not include the demo application and/or optimizations, and instead focus solely on the implementation of the WasmGC backend and runtime system and the performance evaluation. This would allow us to still have a complete implementation of the WasmGC backend and runtime system and a performance evaluation, which are the core contributions of this thesis.

**Runtime Support for Necessary Features** Since implementing the full prelude requries a large amount of work, we will focus our efforts on a subset of the prelude, combinators, and primitives that are necessary for running the benchmarks and building the demo application. We acknowledge that this approach may affect the performance of the generated code (due to it potentially leading to a less efficient implementation of the prelude), but it allows us to focus on the core features of the WasmGC backend and runtime system without getting bogged down in implementing a large number of primitives. If time permits, we can explore adding support for more primitives in the runtime system to improve the performance of the generated code later on.

**Performance Evaluation** With our performance evaluation we aim to answer the following questions:

Q1. How big are the perfomance tradeoffs of using WasmGC when compared to a traditional native backend, and is the performance of WasmGC sufficient for real-world applications taking into account the benefits of using WasmGC such as portability?

Q2. Is it worth targeting WasmGC instead of WasmMVP for Haskell-to-Wasm compilers in terms of performance?

Q3. How well does the WasmGC backend perform in relation to a mainstream Haskell compiler such as GHC, and what does this say about the potential of WasmGC for running real-world Haskell applications?

Q4. How well can the generated Wasm code be optimized and how does this compare to the optimizability of traditional Wasm?

To answer these questions we will run a selected set of benchmarks from the Nofib benchmark suite [27], which is the defacto standard benchmark suite for Haskell compilers. NoFib consists of four sets of benchmark programs: imaginary, spectral, real, and shootout. We will not be able to run the real and shootout sets of benchmarks as they rely on features such as concurrency and foreign function interfaces that will not be implemented in the first version of our WasmGC backend. The imaginary and spectral sets of benchmarks consists of smaller algorithmic programs that do not rely on such features. We will start by running the imaginary benchmarks as they are the simplest benchmarks in terms of features used and then we will move on to the spectral benchmarks if time permits.

Since we are comparing the performance of Wasm binaries to native binaries, we need to ensure that our measurements are fair and consistent across different environments (i.e. Haskell, C, and Wasm/JavaScript). To ensure this, we will perform research in the precision and consistency of different timing and measurement functions available in these environments and select those that are most similar in terms of accuracy and consistency.

Furthermore, when comparing MicroHs produced binaries to GHC produced binaries, we need to ensure that the comparison is fair and that we are comparing the performance of the backends rather than the performance of the runtime systems or optimizations performed by the compilers. To ensure this, we will compare the relative performance of both backends to their respective native backends. This way we can see how much overhead each backend introduces compared to its native counterpart, which gives us insight into the performance of the backends themselves rather than the performance of the runtime systems or optimizations.

**Demo Application** We will develop a small demo application to showcase the capabilities of the WasmGC backend. This could for example be a simple version of the Haskell Playground [28] that runs completely in the browser and allows users to write and execute Haskell code using the WasmGC backend. This would allow us to demonstrate the practical usability of the WasmGC backend and its potential for real-world applications.

**Optimizations** There are multiple potential optimizations we can explore to improve the performance of the WasmGC backend. Some examples are as follows (others may be identified later on during the performance evaluation):

- Removing recursion from the implementation of strict primitives, this can potentially improve performance as shown in the C runtime system of MicroHs. This runtime system makes use of tags which are put on the stack to indicate whether a value is being evaluated to be used by a strict primitive or not, checking for these tags on the stack makes it possible to not have to recurse. Results from the C

runtime system show that this can lead to a 3% performance increase or decrease depending on the hardware it is run on. More importantly every recursion causes a LAS to be allocated increasing memory usage, so removing recursion can lead to memory usage improvements.

- Testing different stack allocation strategies for the LAS to optimize memory usage and performance. We cannot statically determine the necessary stack size for the LAS, so we currently allocate a fixed size array of 100000 elements for the LAS. This may use more memory than necessary for some programs, while for other programs it may not be sufficient. We can explore different strategies for dynamically resizing the LAS or we can try using the Wasm value stack instead of a separate array for the LAS, this however will require very careful management of the value stack.

- Removing unused combinators and primitives from the runtime when a program is compiled. This has multiple benefits. First it can reduce the size of the generated Wasm binary which leads to faster load times. Second it leads to less branches (and thus less integer comparisons to check which combinator/primitive needs to be reduced) in the code responsible for reducing combinators and primitives, which can lead to improved performance. Last it can reduce the attack surface of the program, for example the program access to file access primitives can be removed if the program does not use them, this can prevent potential security vulnerabilities in the runtime system.

**Lightweight Threads** To provide concurrency support in the runtime system we can implement lightweight threads using stack switching [29]. Every thread would be represented by its own stack and the program could switch between these stacks to achieve concurrency. This would involve implementing a scheduler to manage the threads and ensure that they are executed fairly.

**JavaScript Interoperability** Since we are targeting the web, adding support for the Haskell-JavaScript foreign function interface can enable Haskell code to interact with JavaScript code running in the same environment. This would potentially allow features such as DOM manipulation, event handling, and access to browser APIs from Haskell code, which are necessary for building real-world web applications.

# Bibliography


[1] Victor Nazarov, Hamish Mackenzie, and Luite Stegeman. *GHCJS Haskell to JavaScript compiler*. 2023. URL: https://github.com/ghcjs/ghcjs (visited on 12/24/2025).

[2] Andreas Haas et al. "Bringing the web up to speed with WebAssembly". In: *Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation*. 2017, pp. 185–200.

[3] *GHC Wasm Meta Repository*. 2025. URL: https://gitlab.haskell.org/haskell-wasm/ghc-wasm-meta (visited on 12/24/2025).

[4] *Asterius: A Haskell to WebAssembly compiler*. 2022. URL: https://github.com/tweag/asterius (visited on 12/24/2025).

[5] Andreas Rossberg. *WebAssembly 3.0*. 2025. URL: https://webassembly.org/news/2025-09-17-wasm-3.0/ (visited on 12/24/2025).

[6] Lennart Augustsson. "MicroHs: A small compiler for haskell". In: *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium*. 2024, pp. 120–124.

[7] Apoorva Anand. "Towards a WebAssembly Backend for MicroHs". MSc Thesis. Utrecht University, 2025.

[8] Lennart Augustsson. *MicroHs repository*. 2026. URL: https://github.com/augustss/MicroHs (visited on 02/07/2026).

[9] Alon Zakai. "Emscripten: an LLVM-to-JavaScript compiler". In: *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*. 2011, pp. 301–312.

[10] David A Turner. "A new implementation technique for applicative languages". In: *Software: Practice and Experience* 9.1 (1979), pp. 31–49.

[11] Stephanie Weirich. *Lambda-Calculus cooked n-ways*. 2025. URL: https://github.com/sweirich/lambda-n-ways/tree/main (visited on 02/07/2026).

[12] Yorhel. *An Opiniated Survey of Functional Web Development*. May 2017. URL: https://dev.yorhel.nl/doc/funcweb.

[13] Evan Czaplicki. "Elm: Concurrent frp for functional guis". In: *Senior thesis, Harvard University* 30 (2012).

[14] *PureScript*. URL: https://purescript.org/ (visited on 12/27/2025).

[15] *Binaryen: A compiler and toolchain infrastructure library for WebAssembly*. URL: https://github.com/WebAssembly/binaryen (visited on 12/29/2025).

[16] *Support for WebAssembly (Wasm)*. 2025. URL: https://docs.flutter.dev/platform-integration/web/wasm (visited on 12/29/2025).

[17] *Kotlin/Wasm*. URL: https://kotlinlang.org/docs/wasm-overview.html (visited on 12/29/2025).

[18] Léo Andrès, Pierre Chambart, and Jean-Christophe Filliâtre. "Wasocaml: compiling OCaml to WebAssembly". In: (2023).

[19] Andy Wingo. *Guile Hoot*. 2023. URL: `https://gitlab.com/spritely/guile-hoot` (visited on 12/29/2025).

[20] *Getting Started for J2CL/Wasm (Experimental)*. 2025. URL: `https://github.com/google/j2cl/blob/master/docs/getting-started-j2wasm.md` (visited on 12/29/2025).

[21] Manuel Serrano and Robert Bruce Findler. "A Snapshot of the Performance of Wasm Backends for Managed Languages". In: *Proceedings of the 22nd ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes*. 2025, pp. 93–106.

[22] Alon Zakai. *A new way to bring garbage collected programming languages efficiently to WebAssembly*. 2023. URL: `https://v8.dev/blog/wasm-gc-porting` (visited on 12/29/2025).

[23] David A Turner and MS Joy. *SASL language manual*. University of Warwick, Department of Computer Science, 1987.

[24] David A Turner. "The semantic elegance of applicative languages". In: *Proceedings of the 1981 Conference on Functional Programming Languages and Computer Architecture*. 1981, pp. 85–92.

[25] David A Turner. "Miranda: A non-strict functional language with polymorphic types". In: *Conference on Functional Programming Languages and Computer Architecture*. Springer. 1985, pp. 1–16.

[26] Simon L Peyton Jones and Jon Salkild. "The spineless tagless G-machine". In: *Proceedings of the fourth international conference on Functional programming languages and computer architecture*. 1989, pp. 184–201.

[27] Will Partain. "The nofib benchmark suite of Haskell programs". In: *Functional Programming, Glasgow 1992: Proceedings of the 1992 Glasgow Workshop on Functional Programming, Ayr, Scotland, 6–8 July 1992*. Springer. 1993, pp. 195–202.

[28] Tom Smeding. *Haskell Playground*. 2025. URL: `https://github.com/haskell/play-haskell` (visited on 02/07/2026).

[29] Luna Phipps-Costin et al. "Continuing WebAssembly with effect handlers". In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA2 (2023), pp. 460–485.

# Appendix A: Ethics and Privacy QuickScan

## Response Summary:

## Section 1. Research projects involving human participants

**P1. Does your project involve human participants? This includes for example use of observation, (online) surveys, interviews, tests, focus groups, and workshops where human participants provide information or data to inform the research. If you are only using existing data sets or publicly available data (e.g. from X, Reddit) without directly recruiting participants, please answer no.**

- No

## Section 2. Data protection, handling, and storage

The General Data Protection Regulation imposes several obligations for the use of **personal data** (defined as any information relating to an identified or identifiable living person) or including the use of personal data in research.

**D1. Are you gathering or using personal data (defined as any information relating to an identified or identifiable living person )?**

- No

## Section 3. Research that may cause harm

Research may cause harm to participants, researchers, the university, or society. This includes when technology has dual-use, and you investigate an innocent use, but your results could be used by others in a harmful way. If you are unsure regarding possible harm to the university or society, please discuss your concerns with the Research Support Office.

**H1. Does your project give rise to a realistic risk to the national security of any country?**

- No

**H2. Does your project give rise to a realistic risk of aiding human rights abuses in any country?**

- No

**H3. Does your project (and its data) give rise to a realistic risk of damaging the University's reputation? (E.g., bad press coverage, public protest.)**

- No

**H4. Does your project (and in particular its data) give rise to an increased risk of attack (cyber- or otherwise) against the University? (E.g., from pressure groups.)**

- No

**H5. Is the data likely to contain material that is indecent, offensive, defamatory, threatening, discriminatory, or extremist?**

- No

**H6. Does your project give rise to a realistic risk of harm to the researchers?**

- No

**H7. Is there a realistic risk of any participant experiencing physical or psychological harm or discomfort?**

- No

**H8. Is there a realistic risk of any participant experiencing a detriment to their interests as a result of participation?**

- No

**H9. Is there a realistic risk of other types of negative externalities?**

- No

## Section 4. Conflicts of interest

**C1. Is there any potential conflict of interest (e.g. between research funder and researchers or participants and researchers) that may potentially affect the research outcome or the dissemination of research findings?**

- No

**C2. Is there a direct hierarchical relationship between researchers and participants?**

- No

## Section 5. Your information.

This last section collects data about you and your project so that we can register that you completed the Ethics and Privacy Quick Scan, sent you (and your supervisor/course coordinator) a summary of what you filled out, and follow up where a fuller ethics review and/or privacy assessment is needed. For details of our legal basis for using personal data and the rights you have over your data please see the University's privacy information. Please see the guidance on the ICS Ethics and Privacy website on what happens on submission.

**Z0. Which is your main department?**

- Information and Computing Science

**Z1. Your full name:**

Martijn Voorwinden

**Z2. Your email address:**

m.c.b.voorwinden@students.uu.nl

**Z3. In what context will you conduct this research?**

- As a student for my master thesis, supervised by::
  Marco Vassena

**Z5. Master programme for which you are doing the thesis**

- Computing Science

**Z6. Email of the course coordinator or supervisor (so that we can inform them that you filled this out and provide them with a summary):**

m.vassena@uu.nl

**Z7. Email of the moderator (if you are studying a programme from Information and Computing Sciences, please leave the value provided, otherwise change this to the moderator as provided by the coordinator of your thesis project):**

ics-ethics@uu.nl

**Z8. Title of the research project/study for which you filled out this Quick Scan:**

Towards A WasmGC backend and runtime for the MicroHs compiler

**Z9. Summary of what you intend to investigate and how you will investigate this (200 words max):**

We intend to investigate the potential benefits of using WasmGC when compared to WasmMVP in the context of lazy functional programming languages, specifically Haskell. We will do this through extending the MicroHs compiler and runtime to generate and process WasmGC code, benchmarking the resulting binaries, and comparing their performance to native performance and WasmMVP performance.

**Z10. In case you encountered warnings in the survey, does supervisor already have ethical approval for a research line that fully covers your project?**

- Not applicable

---

## Scoring

- Privacy: 0
- Ethics: 0

---